SENTINEL

Talking to the operating system: reading feeds, writing the mission log, precise timestamps

Status and deadline

This prelab is **required**. Every student submits it. It carries **no marks**.

Submit it on Brightspace by midnight on Wednesday, October 7, 2026. The Lab 1 session is on October 9.

Two later assessments use this prelab directly. The oral exam[1] can ask you any question from it. The deliverable[2] is the code that these questions describe, so what you work out here is what you then build.

The deliverable of Lab 1 is marked out of 80, and it gives 4% of the course grade. The oral exam is marked out of 20, and it gives 2%. Together they give 6%.

Instructions

This prelab prepares you for Lab 1, in which SENTINEL writes the mission log.[3]Each question covers one part of the operating system interface that you build during the lab. If you answer these questions before the session, the lab work is easier to understand and quicker to complete.

- Do this prelab individually. Labs are individual submissions. Submit it on Brightspace by midnight on October 7, 2026.
- Read the manual pages named in each question before answering.
- On Ubuntu, if the required manual pages are not installed, install them with:

```
sudo apt install manpages-dev
```

- Write the answers in your own words.

Q1. File descriptors and `open()`

Read:

```
man 2 open
man 2 close
man 2 umask
```

(a)
What is a file descriptor? What are file descriptors 0, 1, and 2 normally used for?

[1]The oral exam of a lab is in the session of the next lab: for Lab 1, the Lab 2 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes it.

[2]The Lab 1 Brief, `docs/lab1/lab1_brief.pdf` in your repository. It states the contract of Lab 1.

[3]The mission log is the file in which SENTINEL records each action, with its time (requirement R6). The Lab 1 Brief, Section 1, `docs/lab1/lab1_brief.pdf`, states what Lab 1 writes to it.

(b)
What does each of the following flags do?

- `O_CREAT`
- `O_TRUNC`
- `O_APPEND`

Why is `O_APPEND` useful for the SENTINEL mission log[4]?

(c)
A process has `umask = 022` and creates a file with mode `0666`. What permissions does the new file have?

Q2. Reading, writing, and errors

Read:

```
man 2 read
man 2 write
man 7 signal
```

(a)
What does `read()` return when it:

1. reads data successfully;
2. reaches the end of the input;
3. encounters an error?

(b)
Can a successful `read()` return fewer bytes than requested? Give one example.

(c)
Can `write()` write fewer bytes than requested? What should a program do if it must write all the requested bytes?

(d)
What does `EINTR` mean? Why should a reliable I/O function normally retry after `EINTR`?

Q3. Timestamps and the mission log

Read:

```
man 2 clock_gettime
```

(a)
Which clock should SENTINEL use to measure how long processing the feed[5] takes: `CLOCK_REALTIME` or `CLOCK_MONOTONIC`? Explain why.

[4]The mission log is the file in which SENTINEL records each action, with its time (requirement R6). The Lab 1 Brief, Section 1, `docs/lab1/lab1_brief.pdf`, states what Lab 1 writes to it.

[5]`demo_lab1` is the demonstration program of Lab 1. It reads a feed, a file of contact reports, and writes the mission log. The Lab 1 Brief, Section 3, `docs/lab1/lab1_brief.pdf`, specifies it.

(b)
The mission log timestamp has the form:

```
1755534061.482913
```

Given:

```
struct timespec ts;
```

write the `snprintf()` call that formats seconds and exactly six digits of microseconds.

(c)
The mission log[6] uses a level field that is 8 characters wide and left aligned, followed by one space. How would you format `INFO` so that the result is aligned correctly?

Q4. What the kernel knows about a process

Read:

```
man 5 proc
```

Run:

```
grep -E '^(Name|Pid|PPid|Threads|VmRSS)' /proc/self/status
ls -l /proc/self/fd
```

(a)
What kind of information does `/proc/[pid]/status` provide? Name two useful fields for SENTINEL.

(b)
What is stored in `/proc/[pid]/fd`? What does each entry represent?

(c)
Why are open file descriptors useful when debugging a program such as SENTINEL? Give one example of a problem they can reveal.

Q5. A reliable write loop

Use your answers from Q2.

Write the function below. It must write all `n` bytes before returning success. A signal that causes `write()` to return `-1` with `errno == EINTR` is not an error and must be retried.

```
int sc_write_all(int fd, const void *buf, size_t n);
```

Your function must:

- return `0` only after all `n` bytes have been written;
- return `-1` on an actual error;
- handle partial writes; and

[6]The mission log is the file in which SENTINEL records each action, with its time (requirement R6). The Lab 1 Brief, Section 1, `docs/lab1/lab1_brief.pdf`, states what Lab 1 writes to it.

- retry `EINTR`.

You may assume the arguments are otherwise valid.

Five questions. No marks. Required.